

Module Name - Software Development Tools & Kits

Student Name - Sakun Arachchi

Student ID – 10965445

Contents

1. Introduction	2
2. Defect Overview	2
3. Defect Identification Context	3
4. Defect Report	3
4.1 Defect Classification	3
4.2 Defect Description	4
4.3 Environment Details	4
4.4 Steps to Reproduce	4
4.5 Expected Result	5
4.6 Actual Result	5
5. Root Cause Analysis	6
6. Risk Assessment	6
7. Security Implications	7
8. Traceability Matrix	7
9. Defect Origin Analysis	7
10. Defect Triage and Resolution	8
11. Preventive Measures	8
12. Glossary	9
13. Conclusion	9

1. Introduction

Quality assurance of software products is the systematic and structured practice aimed at ensuring the compliance with specific quality standards, functional requirements, and user expectations. In contrast to software testing, QA encompasses the whole software development lifecycle (requirement analysis, design, coding, testing, implementation, support). The primary goal of SQA is to prevent defects, not to detect them.

As mentioned above, software testing is just a part of quality assurance processes related to performing certain actions to find defects in the system. SQA process covers a wide range of activities, such as process improvement, enforcing standards, conducting audits, and defect prevention. Effective quality assurance practices provide users with reliable, secure, efficient, and maintainable software.

In this report, one significant defect in the Login page of Enterprise Cloud Suite (version 2.4.0) detected in System Testing is described. Namely, it is the inability of the system to correctly process authentication using Unicode characters. The defect was found during the process of Software testing with the help of such popular techniques as BVA and Negative testing.

This report is created to provide structured documentation of defects found in software using Software Quality Assurance standards.

2. Defect Overview

Upon performing System Testing for the authentication module, a defect has been identified in the Login feature of the application. The issue identified is that the application does not support input with Unicode characters, such as emojis, during password validation. As a result, the application becomes unstable, and the server goes down.

This particular defect is a critical authentication defect because the system login feature is hindered by it. Authentication features are an integral part of software applications; therefore, any defect here is high-severity.

3. Defect Identification Context

Name of System: Enterprise Cloud Suite

Version: v2.4.0

Name of Module: Authentication / Login

Phase of Testing: System Testing

Type of Testing: Function & Negative Testing

Defect ID: ECS-LOGIN-042

The issue found during the testing phase when invalid/boundary values are passed through tests to check system stability and robustness and the application failed while handling Unicode-based password values.

4. Defect Report

4.1 Defect Classification

Defect ID : ECS-LOGIN-042

Title: Authentication Failure due to Unicode/Emoji Password Input

Severity: 1 (Blocker)

Priority: High (P1)

Type: Functional / System Crash

Component: Authentication Module

Status: Open

4.2 Defect Description

This issue occurs when there is an effort to login with a password that uses 4-byte Unicode characters like emojis. Here, the backend authentication service will not be able to comprehend the input given as there is a difference between the character encodings of the backend and frontend services.

Consequently, there will be a creation of runtime exceptions.

4.3 Environment Details

Operating System: Windows 11 / Android 14

Browser: Google Chrome v122+, Microsoft Edge

Server: Staging Environment

Database: MySQL 8.0

Application URL: <https://staging.enterprise.com/login>

4.4 Steps to Reproduce

- 1.Open the login page in a supported browser
- 2.Enter valid username:
qa_tester_99@enterprise.com
- 3.Enter password containing emoji:
SafePass2026☐☐
- 4.Click on the Login button
- 5.Observe system behavior and error logs

4.5 Expected Result

The system should:

- Accept valid Unicode input (including emojis)
- Encode input using UTF-8 standard
- Hash the password securely
- Authenticate the user successfully if credentials are valid
- Display proper error messages if authentication fails
- Prevent exposure of internal system errors

4.6 Actual Result

- The application freezes for 3–5 seconds
- Server returns 500 Internal Server Error
- Exception displayed: java.lang.NullPointerException
- Mobile version crashes unexpectedly
- Internal stack trace is exposed to user

5. Root Cause Analysis

At the heart of this problem is the disparity in character set encoding that exists across different levels of the application.

Front-end application makes use of UTF-8 character encoding

Back-end hash function uses ISO-8859-1 encoding as default

Since emojis need to be encoded using a four-byte UTF-8 encoding, the back end fails to properly decode this string to return a null object that is used in the hash operation and causes a `NullPointerException`.

In addition to that, insufficient input validation, as well as the absence of unicode data in unit tests, contributed to the emergence of this bug.

6. Risk Assessment

The risk calculation formula includes:

Risk level = Severity x Likelihood

Severity = 1 (Blocker)

Likelihood = High

Risk level = 1 x High = Critical Risk level

Impact:

Failure of authentication mechanism

Inconsistent system performance

Unfavorable user experience

Vulnerability due to leak of stack traces

7. Security Implications

This loophole leads to a security flaw that is categorized into the OWASP Top 10 - Security Misconfiguration.

These loopholes include:

Stack trace of Java classes

The internal package architecture

Implementation details on the server side

This information may be used maliciously by attackers to carry out attacks.

8. Traceability Matrix

Test ID	Scenario	Input	Requirement	Status
TC-01	Valid Login	Pass123	REQ-AUTH-01	Pass
TC-02	Boundary Test	2-char password	REQ-AUTH-02	Pass
TC-03	SQL Injection	' OR 1=1 --	REQ-SEC-09	Pass
TC-04	Unicode Input	Emoji 🍌	REQ-AUTH-01	Fail
TC-05	Brute Force	Multiple attempts	REQ-SEC-12	Pass

9. Defect Origin Analysis

Origin Phase: Unit Testing / Code Review

Issue: Missing UTF-8 encoding enforcement

Gap: Lack of Unicode-based test cases

Process Failure: Incomplete validation of non-ASCII input handling

10. Defect Triage and Resolution

Critical defect classification because of severity

Recommended solution:

UTF-8 encoding of all layers of the program.

Change the backend hashing algorithm to:

StandardCharsets.UTF_8

Input validation check during authentication process.

Strict validation of input data by middleware.

Post-fix testing:

Verification testing

Regression testing of authentication/password reset modules

11. Preventive Measures

In order to prevent such a problem from happening again:

Use UTF-8 standardization across all layers of architecture

Include Unicode cases in unit testing

Conduct static analysis in CI/CD process

Enhance exception handling to eliminate stack trace
Do code reviews on the authentication module

12. Glossary

SQA: Software Quality Assurance procedure for checking quality of software

RCA: Root Cause Analysis for identifying source of defects

BVA: Boundary Value Analysis for testing extreme input values

Regression Testing: For ensuring that new modifications do not affect previous functionalities

Test Case: Conditions that check the functionality of the system

13. Conclusion

The ECS-LOGIN-042 errors found in this scenario constitute a major weakness in the functionality of the authentication process in the Enterprise Cloud Suite software application. Although the software runs well when using standard data entry methods, it does not run effectively when presented with new Unicode data entry methods like emojis.

It should be noted that there is an existing problem with regards to the development and testing of software programs, which needs immediate attention.

Proper coding and validation in accordance with the UTF-8 standard will produce effective software.